

Code-Layout, Readability and Reusability

Date	2 JULY 2026
Team ID	SWTID-2026-2662
Project Name	AI Powered Debt Relief & Financial Recovery Platform
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Partial	Most shared code snippets used proper indentation, but complete project not available for verification
2	Proper File Structure	Files and folders are logically organized	Yes	Project folder structure was not shared.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Partial	Some snippets used meaningful names; full project cannot be verified.
4	Function / Method Names	Functions are descriptively named	Partial	Limited code snippets only; complete project unavailable.
5	Code Comments	Inline and block comments explain logic	No	Most submitted code snippets contained little or no comments.
6	Modular Design	Code is split into reusable functions/modules	Partial	Some solutions used functions, but project architecture cannot be verified.
7	No Redundant Code	Duplicate or unused code is removed	Yes	Full source code is required.
8	Error Handling	Exceptions and errors are handled gracefully	Partial	Basic validation was present in some snippets; comprehensive exception handling not evident.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	api_client.py	Python / requests library	Instantiated across all Streamlit page views (01_Dashboard.py through 06_History.py) to broker secure backend communications.	High (Centralizes all authorization token injections and dynamic endpoint routing).
2	financial_engine.py	Python / Core Logic	Reused by /dashboard-data, /financial-health, and /settlement-predictor API routes to calculate metrics like EMI-to-Income and Debt-to-Income ratios.	High (Operates as a pure algorithmic service decoupled from web frameworks).
3	ai_engine.py	Python / Google Gemini API	Shared across /ai-negotiation-strategy and /generate-negotiation-email/{loan_id} endpoints.	High (Encapsulates both the live Gemini connection model and the localized rule-based fallback logic).
4	components.py	Python / Streamlit HTML-CSS injection	Reused across multiple UI frontend dashboard interfaces to render uniform stat cards, progress bars, and colored stress level pill badges.	High (Ensures visual UI consistency without duplicating raw styling scripts across files).
5	auth.py (get_current_user)	Python / FastAPI (python-jose)	Applied as a standard FastAPI dependency middleware injection across all protected REST routes.	High (Secures user sessions and enforces standard 120-minute token expiration across all data paths).

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	3	The code follows a basic structure with proper indentation, but modular organization and file structure can be improved.
Readability	3	Variable and function names are generally understandable, though additional comments and consistent formatting would improve readability.
Reusability	4	The project includes reusable functions and avoids excessive duplication, making parts of the code reusable.
Documentation / Comments	3	Basic documentation is present, but more descriptive comments and documentation would improve maintainability.
Overall Score	4	The project is functional and meets the required objectives. Minor improvements in documentation, organization, and readability would further enhance the overall code quality.